

XL Logic Roadmap

Stand: 2026-05-10

Zielbild

XL Logic soll ein ComputerCraft-ähnlicher Technik-Mod fuer Minecraft 1.21.1 auf NeoForge werden, aber mit Python statt Lua. Der Computer soll ein echtes, serverautoritatives Automationssystem steuern koennen: Netzwerk-Endpunkte erkennen, mit Weltlogik interagieren, Crafting- und Materialfluesse koordinieren und seinen Laufzustand sowohl im UI als auch spaeter auf echten Bildschirm-Blocks sichtbar machen.

Aktueller Stand

- Eine blockweise Statusmatrix mit Einordnung und naechsten Schritten liegt in [docs/BLOCK_STATUS_MATRIX.md](#).
- Toolchain steht: Minecraft 1.21.1, NeoForge 21.1.218, Java 21, GraalPy.
- Projektbuild ist gruen mit `.\gradlew.bat classes`.
- Der NeoForge-GameTest-Server laeuft, und neunzehn automatisierte Kernszenarien fuer Discovery-Konflikt, XLAPI-Segmenttrennung, Farbfilter-Blocker, device_channel-Blocker sowie Editor-Lease-read-only, Viewer-Reopen-Handoff, Recovery-Draft-Resume, Recovery-Draft-Divergenz, Release-Handoff, Timeout-Handoff, Disconnect-Handoff, Reconnect-Reclaim, read-only-Disconnect-Stabilitaet, Teleport-Handoff, Dimensionswechsel-Handoff, Chunk-Unload-Cleanup und Restart-Reload sind mit `.\gradlew.bat runGameTestServer` gruen.
- Der Ingame-Python-Editor ist inzwischen eine brauchbare Arbeitsbasis statt nur ein Minimaeditor: Cursor, Mausplatzierung, Auswahl, Copy/Cut/Paste, Undo/Redo, Auto-Indent, Syntax-Highlighting, lokale Parse-Hinweise und kontextbezogene Suggestions fuer Python-Keyworts, APIs, Seiten, Routen und haeufige State-/Preview-Schluessel sind vorhanden.
- Fuer den Programmier-Einstieg gibt es jetzt eine vereinfachte Beginner-Schicht mit screen-, network-, pause- und repeat-Helfern sowie eine erste englische Einstiegsdoku in [docs/BEGINNER_PROGRAMMING_GUIDE.md](#), waehrend die vollstaendige API weiterhin verfuegbar bleibt.
- Zusaetzlich gibt es jetzt ein echtes Ingame-Guidebook-Item mit Kapitelansicht, Codebeispielen und Erklaerungen fuer Output, Device-Discovery, Schleifen, Redstone, Material-I/O und erste Crafting-Schritte.
- Network Cable, Redstone Bus Cable und die 16 Coloured Redstone Cable-Varianten besitzen jetzt eine erste echte Kabelbasis mit Verbindungsgeometrie, automatischen Anschlusszustanden und serverseitiger Netzlogik; Network Cable traegt lokale Discovery, Redstone Bus Cable transportiert alle 16 Kanaele zwischen Redstone-I/O-Blocken, alle drei Kabelfamilien sind jetzt ueber erste Survival-Rezepte craftbar, und jede Coloured-Variante filtert fest auf genau einen Kanal.
- Computer, Screen, Redstone I/O, Material I/O sowie die erste Crafting-Kette aus Crafting I/O und Crafting CPU besitzen jetzt ebenfalls erste Survival-Rezepte, die bewusst auf den Kabeln und Basiskomponenten aufbauen statt als voneinander isolierte Einzelrezepte zu starten.
- Light Sensor, Clock, Rain Sensor und der XLAPI-Block sind jetzt in dieselbe Survival-Progression eingehaengt: Die Sensoren bleiben fruehe Netzwerkgeraete, waehrend der XLAPI-Block bewusst

erst nach Computer und Netzbasis erreichbar wird.

- Es gibt jetzt zudem einen ersten echten Balancing-Pass fuer diese Progression mit bewusst fairen, relativ guenstigen Rezepten: Screens kommen in Zweierstapeln fuer Multiscreen-Bauten, Sensoren sind leichte Netzwerk-Upgrades statt Materialhuerden, Crafting I/O und Crafting CPU bleiben frueh erreichbar, und auch der XLAPI-Block ist eher ein fairer Netzwerkbaustein als ein spaetes Luxusrezept. Der eigentliche Aufwand des Mods soll im Verstehen, Verkabeln und Programmieren liegen, nicht im Grinden seltener Zutaten.
- Passend zur Survival-Progression gibt es jetzt auch eine erste kleine Advancement-Kette fuer die wichtigsten Meilensteine: Netzbeginn, Signalbus, Computer, Sensorik, Weltanbindung, Screen, Automation und XLAPI-Bruecke.
- Die sichtbare Advancement-Kette wurde ausserdem um konkrete Aufbauziele erweitert: erster Kontrollraum mit platziertem Computer und Screen, eine groessere Monitorwand und eine erste sichtbare Fertigungslinie fuer die Automation.
- Fuer Discovery und Busnetz gibt es jetzt erste direkte Ingame-Debug-Ausgaben: Computer und Network Cable koennen Segmentgroesse, Kabelanzahl, Endpoint-Typen sowie XLAPI-Grenzen und ungeladene Frontier-Kanten ausgeben; Buskabel zeigen Kanalfluss, staerkste Signale, echte Routenhops mit Produzent-/Konsument-Markierungen, Farbfilter-Blocker, seitenseitige Geraetekanal-Konflikte, ungeladene Bus-Frontiers und angebundene Redstone-I/O-Blöcke an.
- Der Computer-Block speichert Script und Runtime-Zustand persistent im BlockEntity.
- Python-Ausfuehrung laeuft serverautoritativ als kooperative Tick-Runtime: F5 startet einen persistenten Python-Run auf dem Computer, F5 stoppt ihn wieder, und lange Schleifen muessen ueber `yield from sleep_ticks(...)` oder `yield from run_loop(...)` zwischen Ticks freiwillig pausieren. Execute-Cooldown, maximale Scriptgroesse, Slice-Watchdog sowie stdout-/stderr-Bytegrenzen bleiben als Runtime-Schutz ueber die Server-Config erhalten.
- Gleichzeitiges Oeffnen desselben Computers nutzt jetzt ein serverseitiges Editor-Lease mit Heartbeat, serverseitigem Logout-, Chunk-Unload-, Teleport- und Dimensions-Cleanup, expliziter Freigabe beim Schliessen, read-only Fremdansicht fuer andere Spieler, restart-sicher rein transientem Lock-Zustand, getesteter Viewer-Reopen-Uebergabe, automatischer Lock-Uebergabe an wartende offene Viewer nach Freigabe, Reichweitenverlust, Dimensionswechsel, Disconnect/Crash oder Timeout sowie expliziten clientseitigen Statusmeldungen bei Zielverlust und Wiederherstellung; bleibt das Ziel laenger unerreichbar, greift jetzt eine feste 10s-Policy mit Auto-Close fuer Viewer und lokalem Recovery-Draft fuer den bisherigen Editor, der anschliessend per Resume-Handshake automatisch wieder an den Computer andockt. Wenn lokaler Draft und serverseitiger Scriptstand dabei auseinanderlaufen, pausiert das Auto-Resume bewusst in einen Konfliktzustand statt still zu ueberschreiben, nutzt eine gemeinsame Basis fuer einen echten Drei-Wege-Merge, uebernimmt nicht konfligierende Server-Hunks automatisch vorab und bietet nur fuer verbleibende Konflikte eine Compare-/Merge-Seitenleiste fuer selektive Server-Uebernahme oder bewusstes Publish des gemergten Drafts.
- Fuer boesartige Busy-Loops und uebergrosse stdout-/stderr-Ausgaben gibt es jetzt gezielte JUnit-Runtime-Regressionstests ueber `.\gradlew.bat test`.
- Gemischte stdout-, stderr-, `show_*` - und Plan-Step-Ausgaben werden jetzt in einer gemeinsamen Laufspur in Emissionsreihenfolge gesammelt.
- Das Netz erkennt lokale benennbare Endpunkte ueber Computer und Kabelpfad; relay-aktive XLAPI-Blocks koennen zusaetzlich entfernte Endpunkte aus anderen geladenen Segmenten derselben Dimension mit gleicher Uplink-Gruppe als bridged Endpunkte importieren.

- XLAPI kann pro Uplink-Gruppe bereits entfernte Computerzustands-Snapshots auflisten, einfache Textnachrichten in eine persistente Mailbox auf dem Zielcomputer zustellen und strukturierte status-, ping-, devices- und runtime-Request-Response-Abläufe über dieselbe Mailbox abbilden.
- Für bridged Gerätetypen gibt es jetzt eine erste explizite Remote-Policy: redstone_io ist read_write, screen, Sensoren, material_io, crafting_io und crafting_cpu sind read_only; Endpoint-Namen und XLAPI-Konfiguration bleiben lokal.
- Pro lokalem Kabelsegment ist aktuell nur ein Computer erlaubt; XLAPI-Blocks trennen gültige Computersegmente voneinander.
- Die Python-API deckt bereits Redstone, Sensoren, Material I/O, Crafting CPU, Crafting I/O und XLAPI-Grundfunktionen ab.
- Material I/O und Crafting CPU tragen echte Inventar-/Fluid-/Crafting-Weltlogik.
- Crafting I/O kann grössere Rezeptgitter, mehrstufige Pläne, benannte Routen und schrittsspezifische Input-/Output-Pfade verwalten.
- Planschritte spiegeln Fortschritt, Teilfehler, Fehler und Skip-Zustände in den Runtime-State zurück.
- Der Computer-Screen kann inzwischen strukturierte Outputs als Zeilen, Key-Value-Blöcke, Tabellen und Plan-Cards rendern und besitzt jetzt eine scrollbare Output-History für den Laufbereich.
- Screen-Blocks können jetzt über verbundene Netzkabel durch den Discovery-Modus des Computers zugeordnet werden, besitzen ein dünnes Frontpanel-Modell mit klarer Vorderseite und animierter Status-LED, rendern den letzten synchronisierten Laufzustand als echtes Front-Panel mit Status-Summary, Zeilen, Key-Value-Karten, Tabellen und Plan-Cards, unterstützen Paging für grössere Inhalte direkt am Block, erlauben einen Fokusmodus mit Detail-Scroll für grosse Tabellen und Cards, reagieren in sichtbaren Cards und Tabellen bereits auf konkrete Feld-, Zeilen- und Spalten-Treffer, bilden für zusammenhängende gleich ausgerichtete, demselben Computer zugeordnete Flächen automatisch rechteckige Multiblock-Screens, skalieren auf grösseren Multiblock-Flächen ihre nutzbare Render- und Trefferfläche jetzt deutlich stärker mit und zerlegen unregelmässige Flächen stabiler in nachvollziehbare Teilrechtecke; ungeladene Controller werden auf Follower-Kacheln explizit benannt. Mehrere Computer auf demselben lokalen Kabelsegment werden dabei als Konflikt behandelt, sofern kein XLAPI-Block die Segmente trennt.

Projektlage auf einen Blick

Meilenstein	Status heute	Kurzfazit
1. Bildschirm- und Terminal-Integration	Vorerst abgeschlossen	Screen-Zuordnung, Frontpanel, LED, echte Rich-Output-Anzeige, Paging, Detail-Fokus inklusive feld-/zeilen-/spaltenbewusster Trefferwahl sowie stabile rechteckige und unregelmässig zerlegte Multiblock-Screens stehen; weitere Interaktionsmodelle und Output-Polishing bleiben vorerst nachgelagerte Ausbaustufen.
2. Netz wirklich fertigziehen	Vorerst abgeschlossen	Discovery-Kabel, Buskabel, Farbfilter, Routing, Boundary-/Blocker-Debug und die ersten automatisierten Kernfälle stehen; grössere Topologien und weitere Netzregeln bleiben bewusste spätere Ausbaustufen.

Meilenstein	Status heute	Kurzfazit
3. Runtime-Haertung und Output-Ordnung	Teilweise umgesetzt	Serverautoritative Ausfuehrung, geordnete Runtime-Ausgaben, konfigurierbares Statement-Budget, Execute-Cooldown, Scriptgroessen-Policy, Watchdog-/Stream-Limits, Editor-Lease, Runtime-Regressionstests, Lease-/Viewer-Reopen-/Recovery-Draft-Resume-/Recovery-Draft-Divergenz-/Chunk-Unload-/Restart-/Teleport-/Dimensions-/Disconnect-/Reconnect-Konflikttests, explizite Zielverlust-/Restore-Hinweise im Client, feste 10s-Auto-Close-/Recovery-Draft-Policy, automatischer Resume-Handshake, expliziter Divergenz-/Konfliktpfad, basisgestuetzter Drei-Wege-Merge mit automatischer Voruebernahme nicht konfligierender Server-Hunks, Compare-/Merge-Seitenleiste mit Hunk-Navigation und navigierbare Output-History stehen; offen sind vor allem weitere Runtime-Kantenfaelle, feinere Konflikt-UX und ggf. weitere abgestufte Policies.
4. Crafting-Netz als echtes Automationssystem	Teilweise umgesetzt	Crafting I/O, CPU, Routen und mehrstufige Plaene sind implementiert; die persistente Plan-Queue kann jetzt zwischen voller Queue-Reservation (full_queue) und nur zyklusweiser Reservation (active_cycle) umschalten, Planschritte liefern explizite Fehlerklassen wie material_missing , buffer_full , output_full , route_missing , recipe_invalid , cpu_unavailable , intermediate_missing und intermediate_contaminated , Queue-Jobs fuehren explizite Gesamtzustaende wie resumable , blocked , failed und completed , exportieren einen strukturierten Job-Snapshot inklusive reservierbarem Zyklus-Praefix und Blockierposition, verfolgen erwartete Zwischenprodukte ueber Buffer-Routen hinweg und geben bei spaeteren Queue-Blockern gezielte Hinweise wie switch_to_active_cycle , aber weitere Skalierung fuer noch groessere Produktionsketten bleibt offen.
5. XLAPI und Multi-Computer-Koordination	Teilweise umgesetzt	Bridged Endpunkte, Remote-Computerlisten, Mailbox und strukturierte Responses sind da; weitergehende Remote-Kommandos, Policy-Haertung und explizite Bridge-Tests fehlen noch.

Meilenstein	Status heute	Kurzfazit
6. Mehrspieler, UX und Abschlussqualitaet	Begonnen	Editor-UX und Doku sind deutlich besser; ein exklusives Editiermodell mit read-only Fremdansicht, automatischer Handoff-Logik, Logout-/Chunk-Unload-/Teleport-/Dimensions-Cleanup, restart-sicher transienten Leases, Viewer-Reopen-Handoff, expliziten Client-Hinweisen, einer festen Verlust-Policy fuer dauerhaft fehlende Ziele, automatischem Recovery-Draft-Resume, explizitem Divergenzpfad, basisgestuetztem Drei-Wege-Merge und Compare-/Merge-Seitenleiste steht, aber weitere Konflikt-UX und Abschlussqualitaet fehlen noch.

Was bereits substantiell umgesetzt ist

1. Fundament

- NeoForge-Grundprojekt und Registries
- erste Blockfamilie inklusive BlockEntities
- Ingame-Editor mit Cursor, Mausplatzierung, Auswahl, Copy/Cut/Paste, Undo/Redo, Auto-Indent, Syntax-Highlighting, lokalen Parse-Hinweisen und kontextbezogenen Suggestions fuer Strings, Routen, Seiten und State-Schluessel
- GraalPy-Integration mit eingeschraenktem HostAccess

2. Computer-Laufzeit

- blockgebundener Computer-Screen
- persistenter Script-Inhalt
- persistenter Runtime-Snapshot
- serverautoritatives F5-Execution-Modell
- konfigurierbares Statement-Budget fuer serverseitige Python-Ausfuehrung
- konfigurierbarer Execute-Cooltdown, maximale Scriptgroesse, Watchdog-Laufzeitlimit und stdout-/stderr-Bytegrenzen als serverseitige Runtime-Guardrails
- gemeinsame Output-Spur fuer stdout, stderr, show_* und Plan-Schritte
- exklusives Editor-Lease mit Heartbeat, read-only Fremdansicht und expliziter Freigabe beim Schliessen
- gezielte Runtime-Regressionstests fuer Busy-Loops und uebergrosse stdout-/stderr-Ausgaben

3. Netz und Endpunkte

- Endpoint-Erkennung und API-Namensvergabe
- Umbenennung von Endpunkten per Namensschild
- PythonExecutionContext aus Netz-Snapshots
- XLAPI als Segmentgrenze und Bridge fuer explizit markierte bridged Endpunkte

4. Welt- und Geraete-API

- Weltzeit, Wetter und Dimensionsdaten
- Redstone-I/O Lesen/Schreiben/Kanaele

- Light Sensor, Clock, Rain Sensor
- Material I/O fuer Items und Fluids
- Crafting CPU mit Recipe-Preview und echter Craft-Ausfuehrung

5. Automationslogik

- Crafting I/O als 3x3-, 5x5- und 7x7-Frontend
- verlinkte Crafting CPU
- Standard-Input- und Output-Routen
- benannte Zwischenlager-Routen
- mehrstufige Plaene mit schrittweiser Ausfuehrung
- pro Schritt eigene Input-/Output-Routen

6. Rich Output Basis

- typisierte Runtime-Outputs statt nur flacher Textzeilen
- Output-Typen: line, key_value, table, plan_card
- Python-Helfer: output.line, output.kv, output.table, output.plan_card
- Komfortfunktionen: show_kv, show_table, show_plan_card
- Rendering im Computer-Screen als Karten und Tabellen

7. Erste Screen-Anbindung

- Screen-Blocks speichern eine explizite Verlinkung zu einem Computer-Block
- Shift-Rechtsklick auf einen Computer startet beziehungsweise aktualisiert Discovery; erreichbare Screens werden ueber das Kabelnetz diesem Computer zugeordnet
- der letzte synchronisierte Computer-Runtime-State wird im Welt-Renderer direkt auf der Vorderseite des Screen-Blocks sichtbar gemacht
- Discovery und serverseitige Skriptausfuehrung verweigern lokale Kabelsegmente mit mehr als einem Computer, solange diese nicht durch XLAPI-Blocks getrennt wurden

Wichtige Luecken

Noch nicht fertig oder nur als Basis vorhanden

- Screen-Block rendert jetzt echte Rich-Output-Elemente direkt in der Welt, kann groessere Inhalte seitenweise durchblaetern, besitzt einen Fokusmodus fuer Detail-Scroll in grossen Tabellen und Cards, waehlt sichtbare strukturierte Ausgaben in der mittleren Frontzone direkt per Trefferposition aus und uebernimmt dabei in Cards die getroffene Feldzeile sowie in Tabellen die getroffene Zeile und den sichtbaren Spaltenbereich, nutzt innerhalb eines fokussierten sichtbaren Ausschnitts eine richtungs- und abstandsabhaengige Navigation statt eines starren Drei-Zonen-Rasters, fasst zusammenhaengende, gleich ausgerichtete Screen-Flaechen eines Computers automatisch zu rechteckigen Multiblock-Panels zusammen, zerlegt unregelmaessige Flaechen stabiler in sichtbare Teilrechtecke und bleibt an Chunk-Grenzen durch Follower-Hinweise besser lesbar; fuer Meilenstein 1 ist dieser Stand vorerst ausreichend, spaetere Ausbaustufen bleiben weitergehende Interaktion und weiteres Output-Polishing.
- Network Cable, Redstone Bus Cable und Coloured Redstone Cable sind fuer Meilenstein 2 jetzt vorerst ausreichend abgeschlossen: Kernfaelle sind per GameTest abgesichert, und weitergehende

Frontier-, Mehrsegment- und Misch-Topologien bleiben bewusste spätere Ausbaustufen statt akute Blocker.

- XLAPI hat jetzt eine erste Computer-zu-Computer-Basis mit Remote-Statuslisten, Mailbox-Nachrichten sowie strukturierten status-, ping-, devices- und runtime-Responses und einer konservativen Remote-Policy pro Gerätetyp, aber es fehlen noch weitergehende schreibende Remote-Kommandos, eine mögliche zweite Policy-Stufe für Material-/Crafting-Geräte und explizite Bridge-Tests.
- Es gibt jetzt Paging, direkte Fokauswahl sichtbarer strukturierter Ausgaben per Trefferposition, feld-/zeilen-/spaltenbewusste Auswahl innerhalb sichtbarer Teilbereiche sowie richtungs- und abstandsabhängige grobe oder feine Navigation innerhalb fokussierter Tabellen und Cards direkt auf dem Screen-Block; reichhaltigere Interaktionsmodelle und weiteres Layout-Polishing bleiben vorerst spätere Ausbaustufen.
- Die lokalen Parse-Hinweise und Suggestions im Editor sind aktuell bewusst heuristisch; ein vollwertiger Python-Parser, semantische Fehlermeldungen und tiefere kontextuelle Vervollständigung fehlen noch.
- Weitere abgestufte Laufzeitpolicies, feinere Konflikt-UX für Recovery-Drafts gegen einen inzwischen veränderten serverseitigen Scriptstand, mögliche feinere Slice-Watchdog-Profile und spätere komfortablere Yield-/Scheduler-Modelle für Hintergrundprogramme fehlen noch.
- Gleichzeitiges Bearbeiten desselben Computers ist jetzt serverseitig exklusiv geregelt, reagiert auf Chunk-Unload, Reichweitenverlust, Dimensionswechsel und Disconnect/Crash mit serverseitigem Cleanup, stellt Leases nach Restart nicht wieder her und ist gegen zentrale Konfliktfälle getestet, aber mögliche spätere Übernahme-/Kooperationsmodelle fehlen noch.
- Höherwertige Job-/Queue-Logik für Crafting-Netze fehlt noch.

Priorisierte Roadmap

Meilenstein 1: Bildschirm- und Terminal-Integration

Status heute: Vorerst abgeschlossen. Screen-Zuordnung über Discovery, persistente Bindung, Frontpanel-Rendering, echte Inworld-Rich-Outputs, Paging für grössere Inhalte, direkte Fokauswahl per Trefferposition inklusive Feld-/Zeilen-/Spaltenwahl, richtungs- und abstandsabhängige Fokusnavigation für grosse Tabellen oder Cards sowie stabile rechteckige und unregelmässig zerlegte Multiblock-Screens stehen; weitergehende Interaktionsstufen und Output-Polishing werden als spätere Ausbaustufen behandelt.

Ziel: Der Screen-Block wird zu einer echten Ausgabeoberfläche für Computer.

Umfang:

- Computer-Output auf gebundene Screen-Blocks spiegeln
- Rich-Output-Blöcke für Inworld-Screens auf sinnvolle Layouts begrenzen und für kleine Screens kompakt halten
- Linking zwischen Computer und einem oder mehreren Screens
- Persistenz und Sync für gebundene Anzeigeinstanzen
- Multiblock-Screens für rechteckige Verbundflächen stabil halten und Irregularitäten weiterhin nachvollziehbar in Teilflächen zerlegen

Done wenn:

- ein Computer seinen Laufzustand auf mindestens einem echten Screen-Block anzeigt
- Text-, Key-Value-, Tabellen- und Plan-Card-Ausgaben auf Screen-Blocks sichtbar sind
- der Zustand nach Chunk-Reload und erneutem Öffnen konsistent bleibt

Meilenstein 2: Netz wirklich fertigziehen

Status heute: Vorerst abgeschlossen. Das physische Netz ist spielbar, die lokalen Discovery-Regeln und Bus-Blocker sind sichtbar und die Kernfaelle laufen sowohl manuell als auch automatisiert; weitere Frontier-, Mehrsegment- und Misch-Topologien bleiben bewusste spätere Ausbaustufen.

Ziel: Platzhalter-Blöcke werden zu einem echten technischen Netz.

Umfang:

- Network Cable als echtes Verbindungsmedium
- Redstone Bus Cable mit verlustfreier Mehrkanal-Weiterleitung
- Coloured Redstone Cable mit Farbtrennung und Bus-Abzweigen
- saubere Trennung zwischen physischem Netz, logischen Kanälen und Endpoint-Discovery

Bereits als Basis vorhanden:

- Network Cable hat jetzt echte Kabelgeometrie und verbindet Computer und Endpunkte über sichtbare Kabelpfade.
- Redstone Bus Cable trägt bereits alle 16 Kanäle zwischen verbundenen Redstone-I/O-Blöcken.
- Die 16 Coloured Redstone Cable-Varianten filtern jeweils fest auf genau einen Kanal, verbinden sich zu benachbarten Coloured-Kabeln nur noch bei gleichem Kanal, bilden damit erste farbgetrennte Bus-Abzweige und lassen sich aus Redstone Bus Cable plus Dye sowie per Umfärben zwischen den Farben herstellen.
- Die aktuelle Discovery-, Segment- und Farbbus-Basis ist jetzt mit reproduzierbaren Ingame-Szenarien in [docs/NETWORK_SCENARIOS.md](#) dokumentiert.
- Eine erste NeoForge-GameTest-Abdeckung prüft bereits lokalen Discovery-Konflikt, XLAPI-Segmenttrennung, Farbfilter-Blocker, device_channel-Blocker sowie Editor-Lease-read-only, Viewer-Reopen-Handoff, Recovery-Draft-Resume, Recovery-Draft-Divergenz, Release-Handoff, Timeout-Handoff, Disconnect-Handoff, Reconnect-Reclaim, read-only-Disconnect-Stabilität, Teleport-Handoff, Dimensionswechsel-Handoff, Chunk-Unload-Cleanup und Restart-Reload automatisiert.

Done wenn:

- Kabel und Busse den Endpunktgraphen wirklich beeinflussen
- Redstone-Kanäle nicht nur lokal am Block, sondern über das Netz routbar sind
- Kernfaelle automatisiert gären laufen; weitergehende Frontier-, Mehrsegment- und Misch-Topologien können später als Ausbau folgen

Meilenstein 3: Runtime-Härtung und Output-Ordnung

Status heute: Teilweise umgesetzt. Editor-UX, lokale Diagnosen, serverautoritative Ausführung, geordnete Runtime-Ausgaben, konfigurierbares Statement-Budget, Execute-Cooldown, Scriptgrößen-Policy, Watchdog-/Stream-Limits, exklusives Editor-Lease, serverseitiger Logout-/Chunk-Unload-/Teleport-/Dimensions-Cleanup, restart-sicher transienter Lease-State, neue Runtime-

Regressionstests, neue Lease-/Viewer-Reopen-/Recovery-Draft-Resume-/Recovery-Draft-Divergenz-/Chunk-Unload-/Restart-/Teleport-/Dimensions-/Disconnect-/Reconnect-Konflikttests, explizite Zielverlust-/Restore-Hinweise im Client, eine feste 10s-Auto-Close-/Recovery-Draft-Policy, automatischer Resume-Handshake, ein expliziter Divergenz-/Konfliktpfad, ein basisgestuetzter Drei-Wege-Merge mit automatischer Voruebernahme nicht konfligierender Server-Hunks, eine Compare-/Merge-Seitenleiste und eine benutzbare Output-Navigation sind vorhanden; offen sind vor allem weitere Runtime-Kantenfaelle, feinere Konflikt-UX und moegliche weitere Policy-Stufen.

Ziel: Laufzeit und UI werden fuer echte Nutzung stabil genug.

Umfang:

- exakte zeitliche Reihenfolge zwischen stdout, stderr und show_* Outputs abbilden
- konfigurierbares Statement-Budget und weitere Guardrails
- Watchdog-Regeln, haertere Laufzeitlimits oder weiter abgestufte serverseitige Policies
- Output-Bereich mit Scrollen, Navigation und ggf. Paging fuer groessere Tabellen

Done wenn:

- gemischte print- und show_* Ausgaben in der Reihenfolge erscheinen, in der das Python-Script sie emittiert hat
- Laufzeitlimits ueber Konfiguration angepasst werden koennen
- derselbe Computer nicht mehr parallel schreibend von mehreren Spielern bearbeitet werden kann, ohne still Zustände zu ueberschreiben
- grosse Tabellen oder viele Output-Cards benutzbar bleiben

Meilenstein 4: Crafting-Netz als echtes Automationssystem

Status heute: Teilweise umgesetzt. Die funktionale Basis ist stark: Eine persistente Plan-Queue mit Schritt-Resume fuer unterbrochene oder partielle Crafting-Plaene steht, Planschritte liefern explizite Fehlerklassen, Queue-Jobs fuehren einen expliziten Gesamtstatus, geblockte Jobs koennen gezielt resumed oder abgebrochen werden, und das System exportiert jetzt auch einen strukturierten Gesamtjob-Snapshot inklusive Zwischenlager-Erwartungen, reservierbarem Zyklus-Praefix, Blockierposition und gezielten Action-Hints. Zwischenlager werden nicht nur auf fehlende, sondern auch auf verunreinigte Zwischenprodukte geprueft, und `rebuild_plan()` leitet Craft-Zahlen jetzt aus gestapelten Eingaben ab und erhaelt bestehende Routenfenster. Insgesamt arbeitet das System damit klarer und robuster als zuvor, aber noch nicht wie ein voll belastbares Produktions-Jobsystem.

Ziel: Vom Rezeptfrontend zur belastbaren Produktionskette.

Umfang:

- persistente Queueing- und Wiederaufnahme-Basis fuer Crafting-Plaene weiter zum echten Jobsystem ausbauen
- Gesamtjob-Entscheidungen und Gesamtplan-Fehler ueber mehrere Queue-Zyklen weiter verfeinern; ein reservierbarer Zyklus-Praefix und Blockierpositionen stehen jetzt bereits im Job-Snapshot
- planbewusste Input-/Output-Konsistenz und jobweite Ressourcenregeln weiter vertiefen; fehlende und verunreinigte Zwischenlager werden jetzt bereits getrennt klassifiziert

- groessere oder heterogenere Rezeptzerlegung weiter ausbauen; `rebuild_plan()` leitet Craft-Zahlen jetzt aus gestapelten Rezeptslots ab und bevorzugt bestehende Planfenster
- Nachverfolgung von Teilproduktionen ueber Zwischenlager hinweg weiter ausbauen; getrackte Zwischenprodukte, `intermediate_missing` und `intermediate_contaminated` stehen jetzt als Basis

Done wenn:

- mehrstufige Plaene nicht nur einmalig feuern, sondern als Jobs nachvollziehbar laufen und ihren Gesamtzustand strukturiert exportieren
- Fehler pro Schritt und Gesamtplan klar unterscheidbar sind
- Materialfluss ueber Zwischenlager im Normalfall stabil bleibt

Meilenstein 5: XLAPI und Multi-Computer-Koordination

Status heute: Teilweise umgesetzt. Remote-Sicht auf Computer und Endpunkte, Mailbox und erste strukturierte Responses sind da; die naechste Ausbaustufe sind klar definierte weitere Kommandos, Tests und belastbare Koordinationsregeln.

Ziel: Mehrere Rechner und Netze sollen sinnvoll zusammenarbeiten.

Umfang:

- Uplink-Gruppen zu echter Netz- oder Nachrichtenkopplung ausbauen
- die bestehende Remote-Policy pro Geraetetyp absichern und bei Bedarf gezielt erweitern
- weitere strukturierte Computer-zu-Computer-Kommandos ueber status/ping hinaus
- klare Grenzen zwischen lokalem und ueber Bridge erreichbarem Netz

Bereits als Basis vorhanden:

- Relay-aktive XLAPI-Blocks importieren entfernte Endpunkte aus anderen geladenen Segmenten derselben Dimension mit gleicher Uplink-Gruppe in den Python-Kontext.
- Diese Endpunkte sind als bridged markiert und behalten Bridge-Metadaten fuer Scripts und UI.
- Pro lokaler XLAPI-Bridge sind bereits Remote-Computerlisten, eine einfache gruppenbasierte Mailbox fuer Textnachrichten und strukturierte status-, ping-, devices- und runtime-Responses verfuegbar.
- Bridged redstone_io ist bereits bewusst read_write; screen, Sensoren, material_io, crafting_io und crafting_cpu sind vorerst read_only, und Umbenennung sowie XLAPI-Rekonfiguration bleiben lokal.
- Die aktuelle Bridge-Policy ist jetzt mit reproduzierbaren Ingame-Szenarien in [docs/BRIDGE_POLICY_SCENARIOS.md](#) dokumentiert.
- Screen-Zuordnung und Multi-Computer-Konfliktpruefung bleiben weiterhin rein lokal pro Kabelsegment.

Done wenn:

- mindestens zwei getrennte Computernetze ueber XLAPI bewusst gekoppelt werden koennen
- die erreichbaren Remote-Endpunkte fuer Python vorhersagbar und dokumentiert sind
- die Remote-Regeln fuer Computer-zu-Computer-Kommunikation, Remote-XLAPI-Konfiguration und Konfliktfaelle explizit festgelegt sind

Meilenstein 6: Mehrspieler, UX und Abschlussqualitaet

Status heute: Begonnen. Ein exklusiver Editor-Lock mit read-only Fremdansicht, automatischer Handoff-Logik, Chunk-Unload-/Teleport-/Dimensions-/Disconnect-/Crash-Cleanup, restart-sicher transienten Leases, Viewer-Reopen-Handoff, expliziten Client-Hinweisen bei Zielverlust und Wiederherstellung sowie einer festen 10s-Auto-Close-/Recovery-Draft-Policy mit automatischem Resume, explizitem Divergenzpfad, basisgestuetztem Drei-Wege-Merge und Compare-/Merge-Seitenleiste steht, und der Editor ist deutlich weiter als zu Beginn; offen bleiben Konflikt-UX und Abschlussqualitaet.

Ziel: Das System ist nicht nur technisch interessant, sondern robust im Spielbetrieb.

Umfang:

- Sperr- oder Kooperationsmodell fuer gleichzeitiges Editieren
- saubere Benutzerhinweise bei Konflikten und Laufzeitfehlern
- weitere Default-Scripts und Doku
- gezielte Testfaelle fuer Netz, Runtime und Crafting

Done wenn:

- Mehrspieler-Konflikte nicht mehr stillschweigend den Zustand ueberschreiben
- Standardablaufe reproduzierbar testbar sind
- README und Roadmap den Ist-Zustand nicht hinter dem Code herziehen

Empfohlene Reihenfolge ab jetzt

1. Meilenstein 3 Rest abschliessen: auf dem basisgestuetzten Drei-Wege-Merge fuer lokale Recovery-Drafts aufbauen und daneben weitere Runtime-Kantenfaelle, feinere Policy-Stufen und Konflikt-UX-Details nachziehen.
2. Danach Meilenstein 4: Crafting-Netz vom guten Funktionsprototypen zum belastbaren Jobsystem machen.
3. Anschliessend Meilenstein 5: XLAPI gezielt um weitere strukturierte Computer-zu-Computer-Kommandos und explizite Bridge-Tests erweitern.
4. Danach Meilenstein 6: Mehrspieler-Konfliktmodell, breite Testabdeckung und Abschlussqualitaet.
5. Spaetere Ausbaustufe: die Netz-GameTests auf Frontier-, Mehrsegment- und Misch-Topologien erweitern, falls weitere Routingregeln wirklich noetig werden.

Kurzfassung fuer Entscheidungen

- Wenn das Ziel zuerst sichtbarer Spielwert ist: als Naechstes Screen-Block-Polishing, Overflow-Bedienung und Inworld-Layouts abschliessen.
- Wenn das Ziel zuerst technische Stabilitaet ist: als Naechstes auf Basis der neuen Runtime-Regressionstests die Mehrspieler- und weiteren Runtime-Kantenfaelle absichern.
- Wenn das Ziel zuerst Automationstiefe ist: nach der Runtime-Haertung direkt Crafting-Jobsystem und danach XLAPI-Koordination ausbauen.

Weiterfuehrende Doku

- Block-fuer-Block-Einschaetzung mit Status, Luecken und naechstem Schritt: [docs/BLOCK_STATUS_MATRIX.md](#)
- Discovery-, Segment- und Farbbus-Abnahme fuer Meilenstein 2: [docs/NETWORK_SCENARIOS.md](#)
- XLAPI-Bridge-Policy und manuelle Ingame-Szenarien: [docs/BRIDGE_POLICY_SCENARIOS.md](#)